1. Importazione della libreria

python

```
import socket
```

- import serve a importare una libreria.
- socket è una libreria Python che permette di creare connessioni di rete tra dispositivi.
- In questo programma viene utilizzata per tentare una connessione TCP alle porte del dispositivo target.

2. Dizionario dei servizi

python

```
SERVIZI = {
   22:   ("SSH", "Accesso remoto sicuro da riga di comando e trasferimento file (SFTP/SCP)"),
   53:   ("DNS", "Risoluzione dei nomi di dominio in indirizzi IP"),
   80:   ("HTTP", "Navigazione web non cifrata"),
   443:  ("HTTPS", "Navigazione web cifrata con TLS/SSL"),
   445:  ("SMB", "Condivisione di file e stampanti in rete (Windows)"),
   3306: ("MySQL", "Database MySQL / MariaDB"),
   3389: ("RDP", "Desktop remoto di Windows"),
   8080: ("HTTP-Alt", "Server web alternativo, proxy o pannelli di amministrazione"),
}
```

Questa parte crea un **dizionario**, cioè una struttura dati che associa una chiave a un valore.

Ogni elemento contiene:

- La porta TCP da controllare.
- Il nome del servizio associato.
- Una breve descrizione del servizio.

Per esempio:

python

```
22: ("SSH", "Accesso remoto sicuro...")
```

Significa che la porta 22 è associata al servizio SSH.

Ecco a cosa corrispondono le porte principali:

Porta	Servizio	Funzione
22	SSH	Accesso remoto sicuro
53	DNS	Risoluzione dei nomi di dominio
80	HTTP	Traffico web non cifrato
443	HTTPS	Traffico web cifrato
445	SMB	Condivisione di file
3306	MySQL	Database
3389	RDP	Desktop remoto
8080	HTTP-Alt	Servizi web alternativi

Il dizionario consente al programma di sapere quali porte testare e cosa mostrare per ciascuna.

3. Impostazione del timeout

python

TIMEOUT = 1

Definisce il tempo massimo di attesa per ogni tentativo di connessione, espresso in secondi.

In questo caso:

- 1 significa un secondo.
- Se la connessione non si conclude entro il tempo previsto, il tentativo può terminare per timeout.

Un timeout breve rende la scansione più rapida, ma può far risultare non accessibili porte che rispondono lentamente.

4. Richiesta dell'indirizzo IP

python

ip = input("Inserisci l'indirizzo IP del dispositivo da scansionare: ")

- input() mostra un messaggio e attende che l'utente inserisca un valore.
- L'utente digita l'indirizzo IP del dispositivo da controllare.

- Il valore inserito viene memorizzato nella variabile ip.

Per esempio, l'utente potrebbe inserire:

text

192.168.1.10

Da questo momento, il programma userà quell'indirizzo per tentare le connessioni.

Attenzione: il codice non verifica esplicitamente che l'indirizzo inserito sia valido.

5. Stampa delle informazioni iniziali

python

print(f"\nScansione del dispositivo: {ip}")

Mostra il messaggio iniziale con l'indirizzo IP inserito.

- print() stampa il testo.
- f"..." è una *f-string*, che permette di inserire il valore di una variabile nel testo.
- \n aggiunge una nuova riga prima del messaggio.

Per esempio:

text

Scansione del dispositivo: 192.168.1.10

La riga successiva è:

python

print("=" * 70)

Stampa 70 volte il carattere = per creare una linea di separazione visiva.

L'operatore *, applicato a una stringa e a un numero intero, ripete quella stringa.

6. Ciclo sulle porte

python

for porta, (servizio, descrizione) in SERVIZI.items():

Questa è una delle righe più importanti.

- for avvia un ciclo.
- SERVIZI.items() restituisce le coppie chiave-valore del dizionario.
- porta riceve il numero della porta.
- servizio riceve il nome del servizio.

- descrizione riceve la descrizione.

Il ciclo ripete le operazioni successive per ogni porta presente nel dizionario.

Nel tuo caso, vengono eseguiti otto tentativi di connessione, uno per ogni porta.

7. Creazione del socket

python

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

Crea un socket, cioè un oggetto utilizzato per comunicare attraverso la rete.

I due parametri indicano:

- socket.AF_INET: utilizza gli indirizzi IPv4.
- socket.SOCK_STREAM: utilizza un socket di tipo TCP.

Quindi il programma prepara una connessione TCP verso una porta del dispositivo.

python

sock.settimeout(TIMEOUT)

Imposta il timeout del socket usando il valore definito precedentemente, cioè un secondo.

Questo evita che un singolo tentativo possa attendere indefinitamente.

8. Gestione del tentativo di connessione

python

try:

try avvia un blocco di codice nel quale potrebbero verificarsi errori.

Il programma prova a eseguire le istruzioni contenute nel blocco. Se si verifica un'eccezione gestita dal successivo except, può mostrare un messaggio d'errore invece di interrompersi immediatamente.

Il tentativo di connessione

python

risultato = sock.connect_ex((ip, porta))

Questa è la riga che effettua materialmente il test della porta.

- sock.connect_ex() tenta di stabilire una connessione TCP.
- (ip, porta) è la coppia che identifica il dispositivo e la porta di destinazione.
- Il risultato viene salvato nella variabile risultato.

In generale:

- 0: la connessione è riuscita.
- Un valore diverso da 0: la connessione non è riuscita.

A differenza di connect(), il metodo connect_ex() restituisce normalmente un codice d'errore anziché generare un'eccezione per ogni errore di connessione.

9. Interpretazione del risultato

python

```
if risultato == 0:
    stato = "APERTA"
else:
    stato = "NON ACCESSIBILE"
```

Qui il programma interpreta il risultato della connessione.

- Se risultato == 0, assegna alla variabile stato il testo "APERTA".
- Altrimenti, assegna "NON ACCESSIBILE".

È importante distinguere i due concetti: **una connessione riuscita conferma che la porta accetta la connessione TCP in quel momento; una connessione fallita non dimostra necessariamente che la porta sia chiusa.** Potrebbe essere filtrata da un firewall, non raggiungibile oppure il servizio potrebbe non rispondere entro il timeout.

10. Stampa dello stato della porta

python

```
print(f"PORTA {porta:5} [TCP] - {stato}")
```

Stampa il numero della porta, il protocollo e lo stato rilevato.

La parte {porta:5} riserva cinque caratteri per il numero, così i risultati risultano allineati.

Esempio:

text

```
PORTA    22 [TCP] - APERTA
```

La riga successiva:

python

```
print(f"    Servizio:    {servizio}")
```

Mostra il nome del servizio associato alla porta.

Per esempio:

text

Servizio: SSH

Poi:

python

print(f" Descrizione: {descrizione}")

Stampa la descrizione del servizio.

Infine:

python

print("-" * 70)

Stampa una linea di separazione lunga 70 caratteri, per rendere più leggibile l'output.

11. Gestione degli errori

python

except socket.error as errore:

Questo blocco gestisce eventuali eccezioni di tipo socket.error generate dalle operazioni eseguite nel blocco try.

python

print(f"PORTA {porta:5} [TCP] - ERRORE: {errore}")

Mostra il numero della porta e il messaggio relativo all'errore.

python

print(f" Servizio: {servizio}")

Mostra il nome del servizio associato alla porta, anche in caso di errore.

python

print("-" * 70)

Aggiunge una linea di separazione.

Nota: gli errori ordinari di connessione, come il rifiuto della connessione, vengono generalmente restituiti da connect_ex() come codici numerici. Per questo motivo non passano necessariamente nel blocco except.

12. Chiusura del socket

python

finally:

    sock.close()

Il blocco finally viene eseguito sia quando il tentativo termina normalmente sia quando si verifica un'eccezione gestita.

- sock.close() chiude il socket.
- Questo permette di liberare le risorse utilizzate per quel tentativo.

È importante chiudere ogni socket prima di passare alla porta successiva.

13. Messaggio finale

python

print("=" * 70)

print("Scansione completata.")

Dopo che il ciclo ha esaminato tutte le porte, il programma stampa una linea di separazione e il messaggio finale.

Significa che tutti i tentativi previsti sono terminati, non necessariamente che tutte le porte siano state identificate con certezza.

Come funziona il programma nel complesso

1. Inserimento dell'indirizzo IP

2. Lettura delle porte dal dizionario

3. Creazione di un socket TCP

4. Tentativo di connessione alla porta

5. Interpretazione del risultato

- Connessione riuscita → APERTA
- Connessione fallita → NON ACCESSIBILE

6. Stampa dei risultati, chiusura del socket e passaggio alla porta successiva